

# Exploring Models for CDC Mortality Data

Team Members: Amy Carr, Emeline Rincker, Aidan Boldea

Course: MATH 537 – Topics in Data Science

Institution: University of Alabama

Date: April 25, 2025

# Contents

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Abstract</b>                                                | <b>3</b>  |
| <b>2</b> | <b>Introduction</b>                                            | <b>3</b>  |
| <b>3</b> | <b>Data</b>                                                    | <b>4</b>  |
| <b>4</b> | <b>Methodology</b>                                             | <b>5</b>  |
| 4.1      | Linear and Polynomial Regression Models for CDC Data . . . . . | 5         |
| 4.2      | Introducing SDoH Data . . . . .                                | 6         |
| <b>5</b> | <b>Results</b>                                                 | <b>7</b>  |
| 5.1      | Observations . . . . .                                         | 8         |
| <b>6</b> | <b>Discussion</b>                                              | <b>9</b>  |
| <b>7</b> | <b>Conclusion</b>                                              | <b>9</b>  |
| <b>8</b> | <b>References</b>                                              | <b>10</b> |
| <b>A</b> | <b>Appendix</b>                                                | <b>11</b> |

# 1 Abstract

Opioid abuse disorders are one of the most pressing public health challenges in the United States. So much so, the issue obtained the title “The Opioid Crisis”. Recent years have shown severe impacts in states such as North Carolina. This project investigates the predictive power of Social Determinants of Health (SDoH), in modeling opioid-related mortality rates, separated by county. We chose both linear and polynomial regression models on the 2018–2022 crude death rates, to predict 2023 rates and compared their performance against a regression model based on a Vulnerability Index derived from principal component analysis (PCA) of the key SDoH indicators. When comparing approaches, the linear regression model for the crude rates had the lowest root mean-squared error on the test data, suggesting that neither higher-degree polynomial models, nor the PCA of SDoH model outperformed the simpler linear approach under current data restrictions. We then discuss possible sources of error, including data suppression and regional reporting limitations, and state that future work might focus on range-based predictions and alternative model types.

# 2 Introduction

Substance use disorder (SUD) remains a critical public health crisis in the United States, with opioid use disorder (OUD) proving to be one of the greatest challenges. In 2022, approximately 76% of drug overdose deaths throughout the country involved opioids. The number of opioid overdoses has increased tenfold since 1999 [1]. Despite the CDC’s recent report of an almost 24% decline in drug overdose deaths in 2024 [2], combating this crisis continues to be a serious public health concern. This report focuses specifically on the state of North Carolina, which has one of the highest opioid misuse rates in the country. Several techniques have been proposed to model this crisis or issues, including [3], [4]. The unique challenge in the modeling of public health issues is the effect of external circumstances. Factors beyond direct medical care affect the health and well-being of a population. Our study aims to investigate the role of Social Determinants of Health (SDoH) in the spread of opioid overdose-related deaths. As described by the U.S. Department of Health and Human Services, SDoH fall into five main categories:

- Economic Stability
- Education Access and Quality
- Health Care Access and Quality
- Neighborhood and Built Environment
- Social and Community Context

These variables can provide proper context to the crisis of opioid overdoses. The purpose of this project is to investigate the extent to which Social Determinants of Health can be used to predict negative opioid outcomes at the county level in North Carolina. The creation of a Vulnerability Index helps public health officials identify regions at increased risk for opioid

mortality and abuse, allowing for proper allocation of scarce resources such as treatment centers and Naloxone.

### 3 Data

Data for this report was pulled from the following sites:

- Source: CDC Wonder Current Multiple Cause of Death Data
  1. Coverage: United States 2018 - 2023
  2. Variables: Each recorded death contains twelve additional variables: location, census region, age group, race, hispanic origin, year of death, month of death, weekday of death, autopsy performed, place of death, and underlying cause of death.
  3. This data set contains pertinent information on specific locations of opioid-related deaths.

In this dataset, “Unreliable” means less than 20 deaths reported, and “Suppressed” means less than 10 deaths reported. To employ mathematical modeling techniques, we had to remove counties with unreliable or suppressed data from the training and testing sets since we did not have numerical data.

- Source: American Community Survey (ACS) Data
  1. Coverage: United States 2018 - 2023
  2. Variables: Health Insurance Coverage, Median Household Income, Employment Status, Housing, and Educational Attainment
  3. This dataset will be used to pull information regarding SDoH by county.
- Source: AIDSVu: County-Level HIV Prevalence Rates
  1. Coverage: United States 2018 - 2023
  2. Description: This file contains information regarding the rate and counts of new HIV cases by year in each county.
- Source: Plotly GeoJSON File of U.S. Counties
  - This file contains geographical data on county boundaries and Federal Information Processing System (FIPS) codes for North Carolina Counties.

Our choice of SDoH data was based on their frequent appearance in existing vulnerability indexes for other public health issues. We examined the multicollinearity of each SDoH variable by calculating the variation inflation factor for each. Since all of the calculated VIF’s were less than 2.5, we keep all of the chosen indicators. The calculated VIF values can be found in Table 1. Finally, these files were joined on County Federal Information Processing System (FIPS) code and year.



| Feature                                          | VIF      |
|--------------------------------------------------|----------|
| Constant                                         | 339.8797 |
| Renter-occupied: 1.51 or more occupants per room | 2.4132   |
| County HIV Rate                                  | 2.5661   |
| Rate: No Health Insurance                        | 1.9395   |
| Percent Less Than HS                             | 1.7878   |
| Percent Some College                             | 1.6534   |
| Percent Unemployed                               | 2.0413   |
| Percent Owner-Occupied Crowded                   | 1.1409   |
| Median Household Income                          | 2.5331   |

Table 1: Variance Inflation Factors (VIF) for Predictor Variables

## 4 Methodology

### 4.1 Linear and Polynomial Regression Models for CDC Data

Our study used linear and polynomial regression modeling methods to construct predictive models for future opioid-related death rates. The descriptive variables were the past five years of data for each county from the CDC Wonder dataset:

|         |                              |
|---------|------------------------------|
| $x_1$ : | crude rate from 1 year ago   |
| $x_2$ : | crude rate from 2 years ago  |
| $x_3$ : | crude rate from 3 years ago  |
| $x_4$ : | crude rate from 4 years ago  |
| $x_5$ : | crude rate from 5 years ago. |

We constructed a matrix  $X$  where the columns represent crude rate data from each year from 2018 to 2022 and the rows represent counties. The vector  $y$  represents the crude rate data for 2023. The data was randomly split into training data (80%) and testing data (20%) to create a linear regression model ( $p = 1$ ) and polynomial regression models ( $p = 2, 3, 4$ , and 8). We removed unreliable and suppressed data so that we had all numerical values with which to train and test the models.

#### Example Python Code for Generating a Regression Model:

```
#model for p=1
poly = PolynomialFeatures(degree = 1)
X_poly = poly.fit_transform(X)
model1 = model.fit(X_poly, y)

coefficients = model1.coef_
intercept = model1.intercept_
terms = [f"({coefficients[i]:.5f})x{i}" for i in range(1, len(coefficients))]
model_string = f"{intercept:.5f} + " + " + ".join(terms)
print(f"Model for p=1: y = {model_string}")
```

This code gives the following output:  
Model for  $p = 1$ :

$$y = 8.71792 + 0.45605x_1 - 0.15675x_2 + 0.34534x_3 + 0.22255x_4 + -0.32570x_5$$

Note: The full Python code for all regression models can be found in the Appendix.

## 4.2 Introducing SDoH Data

We then wished to explore if introducing SDoH data could allow us to predict outcomes with similar or improved accuracy. We pulled eight indicators that are often linked to health outcomes from the ACS DataTables. To determine which indicators were statistically significant, we ran ordinary linear regression. Any indicator with a p-value less than 0.1 was kept. Table 2 is a summary of the original indicators, as well as the regression. Following this

| Indicator                                        | Coefficient | p-value |
|--------------------------------------------------|-------------|---------|
| Constant                                         | 8.7670      | 0.000   |
| Renter occupied: 1.51 or more occupants per room | -0.0001     | 0.022   |
| County HIV Rate                                  | -0.0034     | 0.694   |
| Rate: No Health Insurance                        | 208.8094    | 0.005   |
| Percent Less Than HS                             | 18.8833     | 0.477   |
| Percent Some College                             | -21.8197    | 0.079   |
| Percent Unemployed                               | 306.8475    | 0.002   |
| Owner occupied: 1.51 or more occupants per room  | -80.3107    | 0.697   |
| Median Household Income                          | -0.00003779 | 0.732   |

Table 2: Initial Linear Regression Summary

model, we found four indicator variables to be statistically significant: **Renter occupied: 1.51 or more occupants per room, Rate: No Health Insurance, Percent Some College and Percent Unemployed**. The variables were selected as County Descriptors. In order to assign each county with an individual Vulnerability Index, we ran PCA to project this four-dimensional data onto a one-dimensional subspace. From our previous regression, we found the Percent Some College and Renter occupied: 1.51 or more occupants per room were negatively associated with Crude Rate. As a result, we multiplied each column by  $-1$  before applying PCA. We then scaled each selected column to have a mean of 0, and a standard deviation of 1. We define the Vulnerability Index as

$$\tilde{V} = X \cdot v$$

where  $X$  is the column vector of descriptor data for each county and  $v$  is the principal component vector. The Vulnerability Indexes with more negative values indicating higher vulnerability to negative opioid outcomes. For a more interpretable metric, we also calculated a Vulnerability Score using max-min normalization. This score ranges from 0 to 100, with higher values indicating higher vulnerability to higher opioid overdose rates. The following

equation shows the normalization

$$\text{Vulnerability Score}_i = \left( \frac{\tilde{V}_i - \min(\tilde{V})}{\max(\tilde{V}) - \min(\tilde{V})} \right) \times 100.$$

In order to determine our prediction abilities, we split our Crude Rate data into testing and training subsets using

`sklearn.model_selection.`

80% of the data was used to train a linear regression model, and 20% was left to test. We fit a new linear model using only the Vulnerability Score as an explanatory variable and found the equation.

$$y = 0.4482x + 8.4829.$$

We finally calculated the root mean-squared error of the testing data to quantify our ability to predict Crude Rate.

## 5 Results

Comparing the polynomial models, we found that  $p = 1$  gives the best prediction with a calculated root means-squared error of 4.44. We found that the linear model using Vulnerability Index as the predictor variable had a calculated root means-squared error of 13.08. Table 3 shows the root MSE for each model.

| Past Years' Crude Rate Model | Degree  | Root MSE  |
|------------------------------|---------|-----------|
| Linear Regression            | $p = 1$ | 3.93      |
| Polynomial Regression        | $p = 2$ | 114.75    |
| Polynomial Regression        | $p = 3$ | 131.87    |
| Polynomial Regression        | $p = 4$ | 801.24    |
| Polynomial Regression        | $p = 8$ | 229830.19 |
| Vulnerability Index Model    | Degree  | Root MSE  |
| Linear Regression            | $p = 1$ | 13.08     |

Table 3: Regression Models and their Root Means-Squared Errors

In the vulnerability index model, the average prediction error is approximately 13 opioid overdose deaths per 10,000 total deaths. This is a substantial margin of error, particularly when considered in the context of North Carolina counties, where most observed crude overdose mortality rates fall between 15 and 45 deaths per 10,000. As such, this level of predictive inaccuracy constitutes a significant proportion of the total outcome range and limits the model's effectiveness for precise forecasting or targeted intervention planning.

If we treat the linear regression crude rate model as the "standard" against which we compare the vulnerability index model, we cannot conclude that Social Determinants of Health (SDoH) data improves our prediction of opioid-related death rates.

## 5.1 Observations

### Heat Maps for Crude Rates

Using the pygris and folium packages, we created heat maps to visually compare the areas of highest opioid-related crude death rates and highest vulnerability scores:

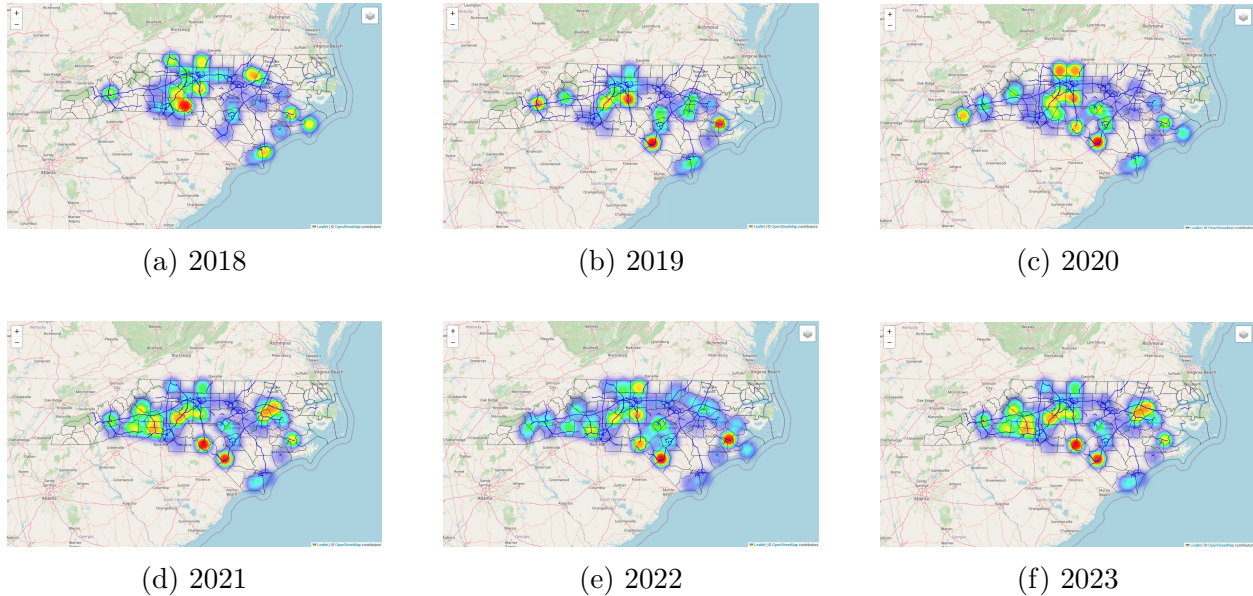


Figure 2: Heat Maps for Crude Rate from 2018 to 2023

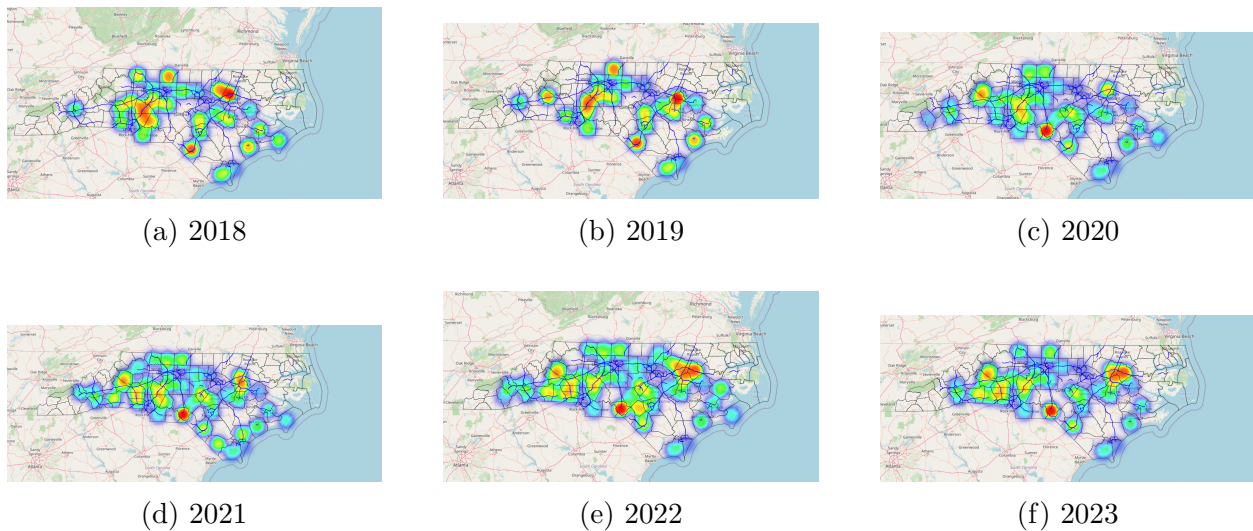


Figure 3: Heat Maps for Vulnerability Score from 2018 to 2023

## 6 Discussion

Our initial goal was to construct a predictive model for opioid crude mortality using county-level social determinants of health data to see if these factors could improve predictions for opioid-related deaths. However, we encountered substantial challenges. Below, we address some of these challenges and offer possible avenues for further research queries.

**Limited Data:** A possible source of the large error in all of the models is that many counties have suppressed or missing data. In particular, low-population and rural areas. This means that for the models based on past years' crude rates, splitting that data into 80% training data and 20% testing data only gives seven test points after removing the "Unreliable" and "Suppressed" data.

**Inaccuracy of the "Standard" Model:** To determine if the model based on SDoH data improved predictions, we constructed a "standard" model based only on prior crude rate data and compared the root MSEs of the models. We recognize that this model is by no means a perfect standard for prediction models, as is evident from a root MSE of almost four, and we recognize that there may be other models that could be used for comparison.

**Other Variables Not Considered:** Our study utilized only four indicator variables in order to predict opioid mortality. In reality, a surplus of other factors are significant in making a county or area more vulnerable to opioid mortality. Future studies may consider opioid dispensing rates, Naloxone administration, as well as sociodemographic data in order to improve forecasting.

**Establishing Ranges Instead of Exact Crude Rates:** Due to the variability of human behavior, an investigation into constructing ranges for crude death rates may be more pertinent to the subject matter. Instead of trying to predict exact values, a study that aims to establish ranges may provide better estimates for resources needed to combat opioid misuse in each area. Another possible approach would be to establish ranges that assign a vulnerability rating (high, medium, low) to certain values.

## 7 Conclusion

After creating and comparing predictive models for crude rates of opioid-related deaths, we did not find that Social Determinants of Health data contributes to improved accuracy in crude rate predictions. Nevertheless, our findings are meaningful as they illustrate the challenges of working with real-world public health data. In particular, we note the unique issues of needing more granular data when privacy is a large concern. We conclude that predictive modeling on this scale is non-trivial and might benefit from improved data collection and reporting.

## 8 References

### References

- [1] Centers for Disease Control and Prevention. (n.d.). *Understanding the opioid overdose epidemic*. Available at: <https://www.cdc.gov/overdose-prevention/about/understanding-the-opioid-overdose-epidemic.html>
- [2] Centers for Disease Control and Prevention. (2025). *CDC reports decline in U.S. drug overdose deaths*. Available at: <https://www.cdc.gov/media/releases/2025/2025-cdc-reports-decline-in-us-drug-overdose-deaths.html#:~:text=New%20provisional%20data%20from%20CDC%27s,activities%20in%20communities%20across%20America.%22>
- [3] Van Handel, M., Rose, C. E., Hallisey, E. J., Kolling, J. L., Zibbell, J. E., Lewis, B., et al. (2016). *County-level vulnerability assessment for rapid dissemination of HIV or HCV infections among persons who inject drugs, United States*. *Journal of Acquired Immune Deficiency Syndromes*, 73(3), 323–331. Available at: [https://journals.lww.com/jaids/abstract/2016/11010/county\\_level\\_vulnerability\\_assessment\\_for\\_rapid.13.aspx](https://journals.lww.com/jaids/abstract/2016/11010/county_level_vulnerability_assessment_for_rapid.13.aspx)
- [4] Kansas Department of Health and Environment. (2022). *2022 Opioid Vulnerability Analysis*. Available at: <https://www.kdhe.ks.gov/DocumentCenter/View/41078/2022-Opioid-Vulnerability-Analysis-PDF>

## A Appendix

### Python Code: CDC Regression Models

```
import numpy as np
import numpy.linalg as LA
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error

#load training and testing data
train_data = pd.read_csv("C:/Users/ecrin/Downloads/train_data80.txt", sep="\t")
test_data = pd.read_csv("C:/Users/ecrin/Downloads/test_data20.txt", sep="\t")

#separate x5 = (5 years prior: 2018), x4 = (4 years prior: 2019), x3 = (3 years prior:
↳ 2020), x2 = (2 years prior: 2021), x1 = (1 year prior: 2022), and y = (predicted
↳ year: 2023)
x5_data = train_data[train_data["Year"] == 2018]
x5 = np.array(x5_data["Crude Rate"])
x4_data = train_data[train_data["Year"] == 2019]
x4 = np.array(x4_data["Crude Rate"])
x3_data = train_data[train_data["Year"] == 2020]
x3 = np.array(x3_data["Crude Rate"])
x2_data = train_data[train_data["Year"] == 2021]
x2 = np.array(x2_data["Crude Rate"])
x1_data = train_data[train_data["Year"] == 2022]
x1 = np.array(x1_data["Crude Rate"])

y_data = train_data[train_data["Year"] == 2023]
y = np.array(y_data["Crude Rate"])

#remove counties(rows) with unreliable/suppressed data, create X matrix
Xy_complete = np.column_stack((x1, x2, x3, x4, x5, y))
    #mask suppressed/unreliable values
Xy_masked = np.ma.masked_where((Xy_complete == "Suppressed") | (Xy_complete ==
↳ "Unreliable"), Xy_complete)
    #remove rows with masked values
Xy = np.ma.compress_rows(Xy_masked)
    #convert values to floats
Xy = Xy.astype(float)
y = Xy[:, -1]
X = Xy[:, :-1]
print(X,y)

#same thing for test data
x5_testdata = test_data[test_data["Year"] == 2018]
x5_test = np.array(x5_testdata["Crude Rate"])
x4_testdata = test_data[test_data["Year"] == 2019]
x4_test = np.array(x4_testdata["Crude Rate"])
x3_testdata = test_data[test_data["Year"] == 2020]
x3_test = np.array(x3_testdata["Crude Rate"])
x2_testdata = test_data[test_data["Year"] == 2021]
x2_test = np.array(x2_testdata["Crude Rate"])
```

```

x1_testdata = test_data[test_data["Year"] == 2022]
x1_test = np.array(x1_testdata["Crude Rate"])

y_testdata = test_data[test_data["Year"] == 2023]
y_test = np.array(y_testdata["Crude Rate"])

Xy_testcomplete = np.column_stack((x1_test, x2_test, x3_test, x4_test, x5_test, y_test))
    #mask suppressed/unreliable values
Xy_testmasked = np.ma.masked_where((Xy_testcomplete == "Suppressed") | (Xy_testcomplete
↪ == "Unreliable"), Xy_testcomplete)
    #remove rows with masked values
Xy_test = np.ma.compress_rows(Xy_testmasked)
    #convert values to floats
Xy_test = Xy_test.astype(float)
y_test = Xy_test[:, -1]
X_test = Xy_test[:, :-1]
print(X_test, y_test)

#define variables
print('x5: crude rate for 2018, x4: crude rate for 2019, x3: crude rate for 2020, x2:
↪ crude rate for 2021, x1: crude rate for 2022')

#define model type
model = LinearRegression()

#model for p=1
poly = PolynomialFeatures(degree = 1)
X_poly = poly.fit_transform(X)
model1 = model.fit(X_poly, y)

coefficients = model1.coef_
intercept = model1.intercept_
terms = [f"({coefficients[i]:.5f})x{i}" for i in range(1, len(coefficients))]
model_string = f"{intercept:.5f} + " + " + ".join(terms)
print(f"Model for p=1: y = {model_string}")

#error for p=1
X_test_poly = poly.fit_transform(X_test)
y_pred = model1.predict(X_test_poly)
MSE1 = mean_squared_error(y_test, y_pred)
RMS1 = np.sqrt(MSE1)
print('root MSE (p=1):', RMS1)

#model for p=2
poly = PolynomialFeatures(degree = 2)
X_poly = poly.fit_transform(X)
model2 = model.fit(X_poly, y)

coefficients = model2.coef_
intercept = model2.intercept_
feature_names = poly.get_feature_names_out(input_features=['x1', 'x2', 'x3', 'x4', 'x5'])

```



```
model_terms = [f"({coefficients[i]:.5f}){feature_names[i]}" for i in
    ↪ range(len(coefficients))]
model_equation = f"{intercept:.5f} + " + " + ".join(model_terms[1:])
print(f"Model for p=2: {model_equation}")

#error for p=2
X_test_poly = poly.fit_transform(X_test)
y_pred = model2.predict(X_test_poly)
MSE2 = mean_squared_error(y_test, y_pred)
RMS2 = np.sqrt(MSE2)
print('root MSE (p=2):', RMS2)

#model for p=3
poly = PolynomialFeatures(degree = 3)
X_poly = poly.fit_transform(X)
model3 = model.fit(X_poly, y)

coefficients = model3.coef_
intercept = model3.intercept_
feature_names = poly.get_feature_names_out(input_features=['x1', 'x2', 'x3', 'x4', 'x5'])
model_terms = [f"({coefficients[i]:.5f}){feature_names[i]}" for i in
    ↪ range(len(coefficients))]
model_equation = f"{intercept:.5f} + " + " + ".join(model_terms[1:])
print(f"Model for p=3: {model_equation}")

#error for p=3
X_test_poly = poly.fit_transform(X_test)
y_pred = model3.predict(X_test_poly)
MSE3 = mean_squared_error(y_test, y_pred)
RMS3 = np.sqrt(MSE3)
print('root MSE (p=3):', RMS3)

#model for p=4
poly = PolynomialFeatures(degree = 4)
X_poly = poly.fit_transform(X)
model4 = model.fit(X_poly, y)

coefficients = model4.coef_
intercept = model4.intercept_
feature_names = poly.get_feature_names_out(input_features=['x1', 'x2', 'x3', 'x4', 'x5'])
model_terms = [f"({coefficients[i]:.5f}){feature_names[i]}" for i in
    ↪ range(len(coefficients))]
model_equation = f"{intercept:.5f} + " + " + ".join(model_terms[1:])
print(f"Model for p=4: {model_equation}")

#error for p=4
X_test_poly = poly.fit_transform(X_test)
y_pred = model4.predict(X_test_poly)
MSE4 = mean_squared_error(y_test, y_pred)
RMS4 = np.sqrt(MSE4)
print('root MSE (p=4):', RMS4)

#model for p=8
poly = PolynomialFeatures(degree = 8)
```

```

X_poly = poly.fit_transform(X)
model8 = model.fit(X_poly, y)

coefficients = model8.coef_
intercept = model8.intercept_
feature_names = poly.get_feature_names_out(input_features=['x1', 'x2', 'x3', 'x4', 'x5'])
model_terms = [f"({coefficients[i]:.5f}){feature_names[i]}" for i in
    ↪ range(len(coefficients))]
model_equation = f"{intercept:.5f} + " + " + ".join(model_terms[1:])
print(f"Model for p=8: {model_equation}")

#error for p=8
X_test_poly = poly.fit_transform(X_test)
y_pred = model8.predict(X_test_poly)
MSE8 = mean_squared_error(y_test, y_pred)
RMS8 = np.sqrt(MSE8)
print('root MSE (p=8):', RMS8)

```

## Python Code: Vulnerability Score Model

```

import pandas as pd
import geopandas as gpd
import numpy as np
import glob
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import statsmodels.api as sm
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

def clean_crude_rate(df):
    df = df[~df['Crude Rate'].isin(['Unreliable', 'Suppressed'])].copy()
    df['Crude Rate'] = pd.to_numeric(df['Crude Rate'])
    return df

# Load file paths through
folder_path = "C:/Users/amy/OneDrive/Desktop/MATH_537/FINALPROJ/Data/Model/"
file_paths = glob.glob(folder_path + "*ModelData.csv")
all_years_df = pd.concat([pd.read_csv(file) for file in file_paths], ignore_index=True)

# Drop the nan values
all_years_df = all_years_df.dropna(subset=[
    'Renter-occupied: 1.51 or more occupants per room',
    'County HIV Cases',
    'County HIV Rate',
    'Crude Rate',
    'Population',
    'Rate: Health Insurance',
    'Rate: No Health Insurance',
    'Percent Less Than HS',
    'Percent Some College',
    'Percent Unemployed',

```

```

        'Percent Owner-Occupied Crowded',
        'Median Household Income'
    ])

# Ensure the County Code variable is cleaned
all_years_df['County Code'] = all_years_df['County Code'].astype(str).str.zfill(5)

'''
PART ONE
We start by running ordinary linear regression in order
to see which indicators are statistically significant.
We will keep any variable with a p-value less than 0.1
'''
filtered_df = clean_crude_rate(all_years_df)

X = filtered_df[
    [
        'Renter-occupied: 1.51 or more occupants per room',
        'County HIV Rate',
        'Rate: No Health Insurance',
        'Percent Less Than HS',
        'Percent Some College',
        'Percent Unemployed',
        'Percent Owner-Occupied Crowded',
        'Median Household Income'
    ]
]
y = filtered_df['Crude Rate']
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print("Regression model summary!")
print(model.summary())

'''
Based on this summary, we find four indicators to be statistically significant
Renter-occupied: 1.51 or more occupants per room
Rate: No Health Insurance
Percent Some College
Percent Unemployed
'''

'''
PART TWO
To calculate our 'vulnerability score', we will use PCA to project the data onto
a 1-D subspace
'''
pca_features = [
    'Renter-occupied: 1.51 or more occupants per room',
    'Rate: No Health Insurance',
    'Percent Some College',
    'Percent Unemployed',
]

vulnerability_dfs = []

```

```

pca_df = all_years_df.dropna(subset=pca_features).copy()
# Since Percent Some College had a negative coefficient in the original linear
→ regression, we flip it in PCA
pca_df['Percent Some College'] = -1*pca_df['Percent Some College']
pca_df['Renter-occupied: 1.51 or more occupants per room'] = -1*pca_df['Renter-occupied:
→ 1.51 or more occupants per room']

# Since some values (i.e. Median Income) are very large, we standardize
# This makes every columns have a mean of 0 and a standard deviation of
scaler = StandardScaler()
X_scaled = scaler.fit_transform(pca_df[pca_features])

'''
How is the index calculated?
We project the data onto the optimal 1-D subspace.
We interpret the vulnerability index as the coordinate on this subspace.
Vulnerability Score =  $X \cdot v$ 
where  $X$  is the column vector for a county and  $v$  is the principal component vector
We chose a 1-D subspace for ease of calculation/interpretation
'''

pca = PCA(n_components=1)
vulnerability_index = pca.fit_transform(X_scaled)
pca_df['Vulnerability Index'] = vulnerability_index
vulnerability_index = (-1)*vulnerability_index
pca_df['Vulnerability Score (0-100)'] = (
    (vulnerability_index - vulnerability_index.min()) /
    (vulnerability_index.max() - vulnerability_index.min())
) * 100

all_years_df = all_years_df.merge(pca_df[['County Code', 'Year', 'Vulnerability Index',
→ 'Vulnerability Score (0-100)']],
    on=['County Code', 'Year'],
    how='left'
)

'''
How to interpret this?
Features with high absolute value contribute more to vulnerability score
The sign of the loading indicates positive or negative influence
'''

pd.DataFrame({
    'Feature': pca_features,
    'Loading': pca.components_[0]
}).sort_values(by='Loading', key=abs, ascending=False)

filtered_df = clean_crude_rate(all_years_df)

'''
PART THREE
We now verify to see how well the vulnerability index predicts opioid mortality
'''

X = filtered_df[['Vulnerability Score (0-100)']]
X = sm.add_constant(X)

```

```

y = filtered_df['Crude Rate']

model = sm.OLS(y, X).fit()
model_summary = model.summary()

model_summary.tables

X = filtered_df[['Vulnerability Score (0-100)']]
y = filtered_df['Crude Rate']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

X_train = sm.add_constant(X_train)
X_test = sm.add_constant(X_test)

model = sm.OLS(y_train, X_train).fit()

y_pred = model.predict(X_test)

'''
Here we plot the differences in actual versus predicted crude rate
'''

plt.scatter(y_test, y_pred)
plt.xlabel("Actual Crude Rate")
plt.ylabel("Predicted Crude Rate")
plt.title("Actual vs Predicted Crude Rate")
plt.show()

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print(model.summary())
print(f"\nTest MSE: {mse:.2f}")
print(f"Test RMSE: {rmse:.2f}")

```

## Python Code: Heat Maps (2018 Crude Rates)

The code for creating the other heat maps is very similar.

```

import folium
import pandas as pd
import pygris
import json5
import geopandas as gpd
from shapely.geometry import Point
from geojson import Point as GeoJSONPoint, Feature, FeatureCollection
from folium.plugins import HeatMap

#2018
# Loading overdose death data for 2018
overdose_data = pd.read_csv("C:/Users/ecrin/Downloads/NC_2018_mortality_data.txt",
    ↪ sep="\t")
overdose_data = overdose_data.dropna(subset=["County Code"])
overdose_data["County Code"] = overdose_data["County
    ↪ Code"].astype(float).astype(int).astype(str).str.zfill(5)

```

```
overdose_data.columns = overdose_data.columns.str.strip()

# Loading county shapefiles for NC
counties = pygris.counties(state="NC", year=2023, cache=True)
counties = gpd.GeoDataFrame(counties)
counties["centroid"] = counties.geometry.centroid
counties["lat"] = counties["centroid"].apply(lambda point: point.y)
counties["lon"] = counties["centroid"].apply(lambda point: point.x)

# Merge overdose data with counties
merged_data = overdose_data.merge(counties[['GEOID', 'lat', 'lon']],
                                   left_on='County Code', right_on='GEOID',
                                   how='left')
merged_data = merged_data.dropna(subset=['lat', 'lon'])
merged_data["Crude Rate"] = pd.to_numeric(merged_data["Crude Rate"], errors="coerce")

# Load NC primary roads
roads = pygris.primary_secondary_roads(state="NC", year=2023, cache=True)
primary_roads = roads[roads["MTFCC"] == "S1100"]

# Create folium map
m = folium.Map(location=[35.25, -80], zoom_start=7)

# Add primary roads to map
folium.GeoJson(primary_roads, name="Primary Roads",
               style_function=lambda x: {"color": "blue", "weight": 1}).add_to(m)

counties = counties.to_crs(epsg=4326)
counties_geojson = counties.drop(columns=["centroid"], errors="ignore").to_json()

# Add county boundaries to the map
folium.GeoJson(counties_geojson,
               name="County Boundaries",
               style_function=lambda x: {"fillOpacity": 0, "color": "black",
                                         "weight": 0.5}).add_to(m)

# Add heatmap layer for overdose data
heat_data = [
    [row["lat"], row["lon"], row["Crude Rate"]]
    for _, row in merged_data.iterrows()
    if not pd.isna(row["Crude Rate"])
]
HeatMap(heat_data, min_opacity=0.2, max_val=max(merged_data["Crude Rate"]),
        radius=25, blur=15).add_to(m)

# Add layer control
folium.LayerControl().add_to(m)

# Display map
m
```